SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA4304**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

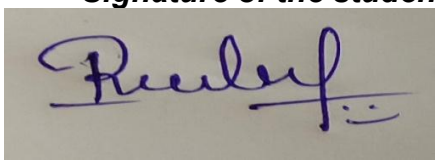
Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I R.venkata.Jashwanth(192411042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Question 1: Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form contains basic elements such as <form>, <input> and <button>. However, it lacks several semantic and accessibility elements.

- <header>: Displays the symposium title and introductory information.
- <main>: Contains the main content of the webpage.
- <section>: Groups related registration content.
- <footer>: Contains copyright and contact information.
- <label>: Describes each input field and improves accessibility.
- <fieldset>: Groups related form controls.
- <legend>: Provides a caption for the fieldset.

These elements improve the webpage's structure, readability, accessibility and maintainability.

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>
  <header>
    <h1>College Online Symposium</h1>
    <p>Register to participate in the symposium.</p>
  </header>
```

```
    <main>
        <section aria-labelledby="registration-heading">
            <h2 id="registration-heading">Registration Form</h2>

            <form action="/register" method="post">
                <fieldset>
                    <legend>Student Details</legend>

                    <p>
                        <label for="name">Full Name:</label>
                        <input type="text" id="name" name="name"
                               autocomplete="name" required>
                    </p>

                    <p>
                        <label for="email">Email Address:</label>
                        <input type="email" id="email" name="email"
                               autocomplete="email" required>
                    </p>

                    <button type="submit">Submit Registration</button>
                </fieldset>
            </form>
        </section>
    </main>

    <footer>
        <p>&copy; College Symposium</p>
    </footer>
</body>
</html>
```

Explanation: The DOCTYPE declaration identifies the document as HTML5. The lang attribute specifies the language, while the character encoding and viewport settings support correct text display and responsive layouts. Semantic elements, labels and required-field validation improve accessibility and usability.

3. Interpret the HTTP request generated during form submission.

When the student clicks the Submit button, the browser collects the form values and sends an HTTP POST request to the /register endpoint.

Example:

POST /register HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

name=StudentName&email=student%40example.com

The request contains the following components:

1. POST: Submits form data to the server.
2. Request URL: /register identifies the server endpoint.
3. Host: Identifies the destination server.
4. Content-Type: Specifies the format of the submitted data.
5. Request body: Contains the student's name and email address.

Unlike GET, POST normally sends form values in the request body instead of the URL query string. The server can validate and process the submitted information and store the registration if appropriate.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle involves the following steps:

1. The student fills in the registration form and submits it.
2. The browser sends an HTTP POST request to the server.
3. The server receives the request at the /register endpoint.
4. The server validates the submitted name and email address.
5. If the details are valid, the server processes the registration and may store it in a database.

6. The server returns an HTTP response containing a status code, headers and possibly a response body.
7. The browser displays a confirmation page or an error message.

For example, a successful response may be:

HTTP/1.1 200 OK

Content-Type: text/html

Alternatively, the server may return a redirect:

HTTP/1.1 303 See Other

Location: /registration-success

The browser then requests the redirected page and displays the result. Thus, HTTP follows a request-response communication model between the browser and server.

5. Suggest improvements for usability and accessibility.

The following improvements can make the registration portal more accessible and user-friendly:

1. Provide visible labels for all form controls.
2. Use the required attribute for mandatory fields.
3. Use type="email" for basic email validation.
4. Use meaningful autocomplete values such as name and email.
5. Display clear instructions and specific error messages near the relevant fields.
6. Support keyboard navigation and maintain a logical tab order.
7. Provide sufficient colour contrast and readable font sizes.
8. Avoid relying only on colour to indicate errors.
9. Use responsive design for mobile phones and desktop computers.
10. Preserve user-entered values when validation fails.

These improvements make the portal easier to use for a wider range of students.

Question 2: Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

The given HTML snippet contains incomplete elements:

```
  <h1>Welcome
  <p>About us
</div>
```

The problems are:

- The <h1> element does not have an explicit closing </h1> tag.
- The element does not have an explicit closing tag.
- The document lacks the <!DOCTYPE html> declaration.
- The <html> element does not specify a language.
- The document lacks character encoding and viewport metadata.

A corrected version of the problematic portion is:

```
<div>
  <h1>Welcome</h1>
  <p>About us</p>
</div>
```

Explicit closing tags and proper nesting make the document structure clearer and easier for browsers, validators and assistive technologies to interpret.

2. Analyze how different browsers may interpret this structure.

HTML parsers are designed to recover from certain markup errors. When closing tags are missing, browsers may automatically close elements or construct a corrected Document Object Model (DOM).

However, relying on automatic error recovery can lead to unexpected results.

- Chrome: Attempts to repair the malformed HTML and display the content.
- Firefox: Constructs a DOM using its HTML parsing rules.
- Microsoft Edge: Recovers from incomplete markup and renders the page.
- Safari: Also attempts to repair the HTML, but rendering differences may occur.

Malformed HTML can affect the DOM structure, CSS styling and JavaScript behaviour. Differences may also depend on browser versions and other code. Therefore, valid and well-structured HTML is important for consistent rendering.

3. Identify missing semantic elements.

The given webpage uses a generic instead of meaningful structural elements.

The appropriate semantic elements are:

- <header>: Contains the page title or introductory content.
- <main>: Represents the primary content of the webpage.
- <section>: Groups related content, such as the company introduction.
- <footer>: Contains copyright information and contact details.

These elements help developers and assistive technologies understand the purpose of each part of the webpage.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section aria-labelledby="about-heading">
      <h2 id="about-heading">About Us</h2>
      <p>Learn more about our company and services.</p>
    </section>
  </main>

  <footer>
    <p>&copy; Company</p>
  </footer>
</body>
</html>
```

Explanation: This code contains a valid HTML5 document declaration, proper nesting, explicit closing tags, language information, metadata and semantic elements. It follows standard HTML document structure and is easier to maintain and validate.

5. Suggest techniques to ensure cross-browser compatibility.

1. Write valid, standards-compliant HTML and CSS.
2. Validate the code during development.
3. Test the webpage in Chrome, Firefox, Edge and Safari.
4. Use responsive CSS and media queries.
5. Avoid obsolete elements and unsupported browser-specific features.
6. Use feature detection when JavaScript functionality depends on browser capabilities.
7. Keep browsers and testing tools updated.
8. Use progressive enhancement so basic content remains usable without advanced features.
9. Test keyboard navigation and accessibility across browsers.
10. Inspect the DOM and computed CSS styles when diagnosing layout differences.

These techniques reduce browser inconsistencies and help provide a consistent user experience.

Question 3: Form Submission Failure

1. Identify issues in the form configuration.

The given form is:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

The main issues are:

1. Incorrect button type: type="button" does not submit the form automatically.
2. Inappropriate HTTP method: GET places submitted values in the URL query string.

3. Security risk: Using GET for login credentials can expose passwords through browser history, server logs and other URL-handling systems.
4. Missing labels: The input fields do not have associated labels.
5. Missing validation: The form does not specify that the username and password are required.
6. Transport security: The application should use HTTPS to protect credentials during transmission.

2. Analyze why the HTTP request is not triggered.

The immediate reason for the failure is the button declaration:

<button type="button">Login</button>

A button with type="button" is a general-purpose button. It does not trigger normal form submission when clicked.

To correct this issue, use:

<button type="submit">Login</button>

When the student clicks this button, the browser initiates form submission, provided that the form passes applicable validation and no script or other condition prevents submission.

JavaScript can also submit a form explicitly, but a normal submit button is the simplest solution when custom behaviour is unnecessary.

3. Interpret the role of the GET method in this scenario.

GET is primarily used to retrieve information from a server. When a form uses GET, the browser appends its submitted values to the URL as query parameters.

For example:

GET /submit?username=student&password=example HTTP/1.1

This is unsuitable for login credentials because URLs may be stored in browser history, server logs, proxy logs and analytics systems. They may also be copied or shared accidentally.

Therefore:

- GET: Suitable for non-sensitive searches, filters and retrieval-oriented forms.
- POST: Appropriate for submitting login credentials in the request body.

However, POST alone does not encrypt information. HTTPS is required to protect credentials during transmission.

4. Propose a corrected implementation.

<!DOCTYPE html>

```
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Student Login</title>
</head>
<body>
   <main>
      <section aria-labelledby="login-heading">
         <h1 id="login-heading">Login</h1>

         <form action="/submit" method="post">
            <p>
               <label for="username">Username:</label>
               <input type="text" id="username"
                     name="username"
                     autocomplete="username" required>
            </p>
            <p>
               <label for="password">Password:</label>
               <input type="password" id="password"
                     name="password"
                     autocomplete="current-password" required>
            </p>
            <button type="submit">Login</button>
         </form>
      </section>
   </main>
</body>
</html>
```

Explanation of corrections:

- method="post" sends the form values in the request body.
- type="submit" triggers normal form submission.
- <label> elements improve accessibility.
- required performs basic browser-side validation.
- autocomplete helps browsers identify username and password fields.

The server must implement the /submit endpoint to receive and authenticate the submitted credentials.

5. Suggest improvements for secure data transmission.

The following security measures should be implemented:

1. Use HTTPS: Encrypts information exchanged between the browser and server.
2. Use POST for credentials: Avoids placing passwords in the URL.
3. Validate input on the server: Never rely only on browser-side validation.
4. Hash passwords securely: Use a strong password-hashing algorithm such as Argon2id or bcrypt instead of storing passwords in plain text.
5. Implement secure session management: Create and manage sessions safely after authentication.
6. Protect session cookies: Use appropriate Secure, HttpOnly and SameSite attributes.
7. Prevent automated attacks: Apply rate limiting and controls against repeated login attempts.
8. Avoid exposing sensitive information: Do not place passwords in URLs, application logs or error messages.
9. Maintain server security: Update frameworks and dependencies and configure appropriate security headers.

The browser form is only one part of the security system. HTTPS, server-side validation, password storage and session management must work together to protect user accounts.

Conclusion

HTML5 semantic elements provide a meaningful structure for webpages, while valid HTML improves accessibility and cross-browser compatibility. Understanding HTTP request-response behaviour helps developers build reliable form submissions. Correct form controls, appropriate HTTP methods and secure transmission practices are essential for creating user-friendly and secure web applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand